

Escaping the Code Maze

The Closed-by-Default Principle















“ The Closed-by-Default Principle removes dead ends from your mental maze of code, revealing a straighter path to understanding.

```

1 <?php
2
3 class Dog
4 {
5     const SOUND = 'Woof';
6
7     public $name;
8
9     public $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     public function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch($item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }

```

Dog could have child classes

SOUND could be of any type

SOUND could be read from the outside

\$name and **\$microchipId** could be of any type

\$name and **\$microchipId** could be read from the outside

\$name's and **\$microchipId**'s values could change during execution

\$name's and **\$microchipId**'s values could be changed from the outside

\$name and **\$microchipId** could be passed any type in the constructor

bark and **fetch** could be called from the outside

bark and **fetch** could return anything

\$item could be passed any type in **fetch**'s signature

PHP 5.0

public / **protected** / **private** access modifiers

final classes

Class and interface type-hinting

PHP 5.0

```
1 <?php
2
3 class Dog
4 {
5     const SOUND = 'Woof';
6
7     public $name;
8
9     public $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     public function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch($item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53 return $item;
```

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53 return $item;
```

PHP 5.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
```

~~Dog could have child classes~~

SOUND could be of any type

SOUND could be read from the outside

\$name and \$microchipId could be of any type

~~\$name and \$microchipId could be read from the outside~~

\$name's and \$microchipId's values could change during execution

~~\$name's and \$microchipId's values could be changed from the outside~~

\$name and \$microchipId could be passed any type in the constructor

~~bark could be called from the outside~~

fetch could be called from the outside

bark and fetch could return anything

~~\$item could be passed any type in fetch's signature~~

PHP 7.0

Scalar type hints and return type declarations

PHP 7.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
28
29 }
30
31 $item = new Ball('Red');
32 $dog = new Dog('Fido', 12345);
33 $dog->fetch($item);
34 $dog->bark();
```

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct(string $name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item): Ball
23     {
24         $this->bark();
25         return $item;
26     }
27 }
28
29 }
30
31 $item = new Ball('Red');
32 $dog = new Dog('Fido', 12345);
33 $dog->fetch($item);
34 $dog->bark();
```

PHP 7.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct(string $name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item): Ball
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
```

SOUND could be of any type

SOUND could be read from the outside

\$name could be of any type ?

\$microchipId could be of any type

\$name's and **\$microchipId**'s values could change during execution

\$name could be passed any type in the constructor ?

\$microchipId could be passed any type in the constructor

fetch could be called from the outside

bark could return anything

fetch could return anything

PHP 7.0

Scalar type hints and return type declarations

declare(strict_types=1) directive

PHP 7.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct(string $name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item): Ball
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53
54 letfn $f($item):
55     $this->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
51 }
52 }
53
54 letfn $f($item):
55     $this->bark();
```

PHP 7.0

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

SOUND could be of any type

SOUND could be read from the outside

\$name could be of any type

\$microchipId could be of any type

\$name's and **\$microchipId**'s values could change during execution

~~**\$name** could be passed any type in the constructor~~

\$microchipId could be passed any type in the constructor

fetch could be called from the outside

bark could return anything

~~**fetch** could return anything~~

PHP 7.1

Class constant visibility

Nullable and **void** types

PHP 7.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
31 }
32 return $item;
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
31 }
32 return $item;
```

PHP 7.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

SOUND could be of any type

~~**SOUND** could be read from the outside~~

\$name could be of any type

\$microchipId could be of any type ?

\$name's and **\$microchipId**'s values could change during execution

~~**\$microchipId** could be passed any type in the constructor~~

fetch could be called from the outside

~~**bark** could return anything~~

PHP 7.4

Typed properties

PHP 7.4

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
50 }
```

```
58 }
```

```
51         return $item;
```

```
50         $this->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private string $name;
10
11     private ?int $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
50 }
```

```
58 }
```

```
51         return $item;
```

```
50         $this->bark();
```

PHP 7.4

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private string $name;
10
11     private ?int $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
30 }
31 }
```

SOUND could be of any type

~~**\$name** and **\$microchipId** could be of any type~~

\$name's and **\$microchipId**'s values could change during execution

fetch could be called from the outside

PHP 8.0

Constructor property promotion

PHP 8.0

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private string $name;
10
11     private ?int $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
50 }
58 }
59 letfn $ifem:
60     $this->park();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         private string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfn $ifem:
56     $this->park();
```

PHP 8.1

readonly properties

PHP 8.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         private string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $fifew:
57 $fifew->bark():
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $fifew:
57 $fifew->bark():
```

PHP 8.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

SOUND could be of any type

~~**\$name**'s and **\$microchipId**'s values could change during execution~~

fetch could be called from the outside

PHP 8.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

SOUND could be of any type

\$name could be read from the outside

fetch could be called from the outside

PHP 8.2

readonly classes

PHP 8.2

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnU $fitem:
57 $this->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnU $fitem:
57 $this->bark();
```

PHP 8.3

Typed class constants

PHP 8.3

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $f:
57 $f->bark():
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $f:
57 $f->bark():
```

PHP 8.3

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

~~SOUND~~ could be of any type

\$name could be read from the outside

fetch could be called from the outside

Before

Dog could have child classes

SOUND could be of any type

SOUND could be read from the outside

\$name and **\$microchipld** could be of any type

\$name and **\$microchipld** could be read from the outside

\$name's and **\$microchipld**'s values could change during execution

\$name's and **\$microchipld**'s values could be changed from the outside

\$name and **\$microchipld** could be of any type in the constructor's signature

bark and **fetch** could be called from the outside

bark and **fetch** could return anything

\$item could be of any type in **fetch**'s signature

After

\$name could be read from the outside

fetch could be called from the outside

Other Relevant Features

PHP 5.3: late static bindings (**::static**)

PHP 7.1: **iterable** type

PHP 7.4: limited covariance / contravariance

PHP 8.0: named arguments, union types, attributes

PHP 8.1: enums, intersection types, **never** return type, **final** class constants

PHP 8.2: Disjunctive Normal Form (DNF) types

PHP 8.3: **#[Override]** attribute

PHP 8.4: asymmetric visibility, property hooks

PHP 8.5: **final** promoted properties

PHP 9.0: dynamic properties will result in a fatal error (unless using **#[AllowDynamicProperties]** attribute)

Static Analysis

Checking the code without running it

Static Analysis

Rector for refactors

<https://getrector.com>

PHP CS Fixer to enforce code-styling rules

<https://cs.symfony.com>

PHPStan to report remaining issues

<https://phpstan.org>

Rector

Type Declarations ruleset

Add missing type hints, property types, constant types, and return types

Rector\Set\ValueObject\SetList::TYPE_DECLARATION

<https://getrector.com/find-rule?rectorSet=core-type-declarations&activeRectorSetGroup=core>

Rector

ClassPropertyAssignToConstructorPromotionRector rule

<https://getrector.com/rule-detail/class-property-assign-to-constructor-promotion-rector>

Part of **PHP 8.0** ruleset **Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_80**

<https://getrector.com/find-rule?rectorSet=php-php-80&activeRectorSetGroup=php>



Rector has rulesets for every PHP version

E.g. `Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_85`

Rector

ReadOnlyPropertyRector rule

<https://getrector.com/rule-detail/read-only-property-rector>

Part of **PHP 8.1** ruleset **Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_81**

<https://getrector.com/find-rule?rectorSet=php-php-81&activeRectorSetGroup=php>

Rector

ReadOnlyClassRector rule (**final** classes only)

<https://getrector.com/rule-detail/read-only-class-rector>

Part of **PHP 8.2** ruleset **Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_82**

<https://getrector.com/find-rule?rectorSet=php-php-82&activeRectorSetGroup=php>



Stricter code



Better static analysis

Rector

Swiss Knife (**finalize-classes**)

<https://github.com/rectorphp/swiss-knife>

Bypass Finals (**DG\BypassFinals::enable();**)

<https://github.com/dg/bypass-finals>

PHP CS Fixer

modifier_keywords rule (part of **@PER-CS** ruleset)

https://cs.symfony.com/doc/rules/class_notation/modifier_keywords.html

protected_to_private rule

https://cs.symfony.com/doc/rules/class_notation/protected_to_private.html

declare_strict_types rule 🌶️

https://cs.symfony.com/doc/rules/strict/declare_strict_types.html

PHPStan

Level 6

Report missing type hints, property types, and return types

<https://phpstan.org/user-guide/rule-levels>

Type Coverage

Report missing constant types (**constant: 100**)

<https://github.com/TomasVotruba/type-coverage>

PHPStan



PHPDoc Types

<https://phpstan.org/writing-php-code/phpdoc-types>

PHPStan

```
1 function add(array $numbers): int
2 {
3     return array_reduce(
4         $numbers,
5         fn (int $total, int $number) => $total + $number,
6     );
7 }
```

```
1 }
2 ):
3     fn (int $total, int $number) => $total + $number,
```

```
1 /** @param array<int,int> $numbers */
2 function add(array $numbers): int
3 {
4     return array_reduce(
5         $numbers,
6         fn (int $total, int $number) => $total + $number,
7     );
8 }
```

```
8 }
1 ):
2     fn (int $total, int $number) => $total + $number,
```

PHPStan

```
1 function add(array $numbers): int
2 {
3     return array_reduce(
4         $numbers,
5         fn (int $total, int $number) => $total + $number,
6     );
7 }
```

```
1 }
2 ):
3     fn (int $total, int $number) => $total + $number,
```

```
1 /** @param list<int> $numbers */
2 function add(array $numbers): int
3 {
4     return array_reduce(
5         $numbers,
6         fn (int $total, int $number) => $total + $number,
7     );
8 }
```

```
8 }
1 ):
2     fn (int $total, int $number) => $total + $number,
```

PHPStan

```
1 function add(array $numbers): int
2 {
3     return array_reduce(
4         $numbers,
5         fn (int $total, int $number) => $total + $number,
6     );
7 }
```

```
1 }
2 ):
3     fn (int $total, int $number) => $total + $number,
```

```
1 /** @param non-empty-list<int> $numbers */
2 function add(array $numbers): int
3 {
4     return array_reduce(
5         $numbers,
6         fn (int $total, int $number) => $total + $number,
7     );
8 }
```

```
8 }
1 ):
2     fn (int $total, int $number) => $total + $number,
```

PHPStan

```
1 /** @param array{'items': list<array{'item': string, 'quantity': int}>, 'codes': list<string>} $numbers */
2 function validate(array $cart): bool
3 {
4     // ...
5 }
```

```
2 }
```

```
4 // ...
```

PHPStan



Generics By Examples

<https://phpstan.org/blog/generics-by-examples>

PHPStan

```
1 class Box
2 {
3     private $item;
4
5     public function __construct($item = null)
6     {
7         $this->item = $item;
8     }
9
10    public function open()
11    {
12        return $item;
13    }
14 }
```

PHPStan

```
1 class Box
2 {
3     private $item;
4
5     public function __construct($item = null)
6     {
7         $this->item = $item;
8     }
9
10    public function open()
11    {
12        return $item;
13    }
14 }
```

```
14 }
13 }
15 return $item;
```

```
1 /** @template T */
2 class Box
3 {
4     /** @var T */
5     private $item;
6
7     /** @param T|null $item */
8     public function __construct($item = null)
9     {
10         $this->item = $item;
11     }
12
13     /** @return T|null */
14     public function open()
15     {
16         return $item;
17     }
18 }
```

```
18 }
17 }
19 return $item;
```

PHPStan

```
1 class Box
2 {
3     private $item;
4
5     public function __construct($item = null)
6     {
7         $this->item = $item;
8     }
9
10    public function open()
11    {
12        return $item;
13    }
14 }
```

```
14 }
13 }
15 return $item;
```

```
1 /** @template T */
2 class Box
3 {
4     /** @var T */
5     private $item;
6
7     /** @param T|null $item */
8     public function __construct($item = null)
9     {
10         $this->item = $item;
11     }
12
13     /** @return T|null */
14     public function open()
15     {
16         return $item;
17     }
18 }
```

```
18 }
17 }
19 return $item;
```

```
1 /** @var Box<string> */
2 $box = new Box('foo');
```

Related Principles

Encapsulation

YAGNI

Immutability

KISS

Open/Closed Principle

Tell, Don't Ask

Principle of Least Privilege

Open/Closed Principle

“ Software entities should be open for extension but closed for modification.

Closed-by-Default Principle

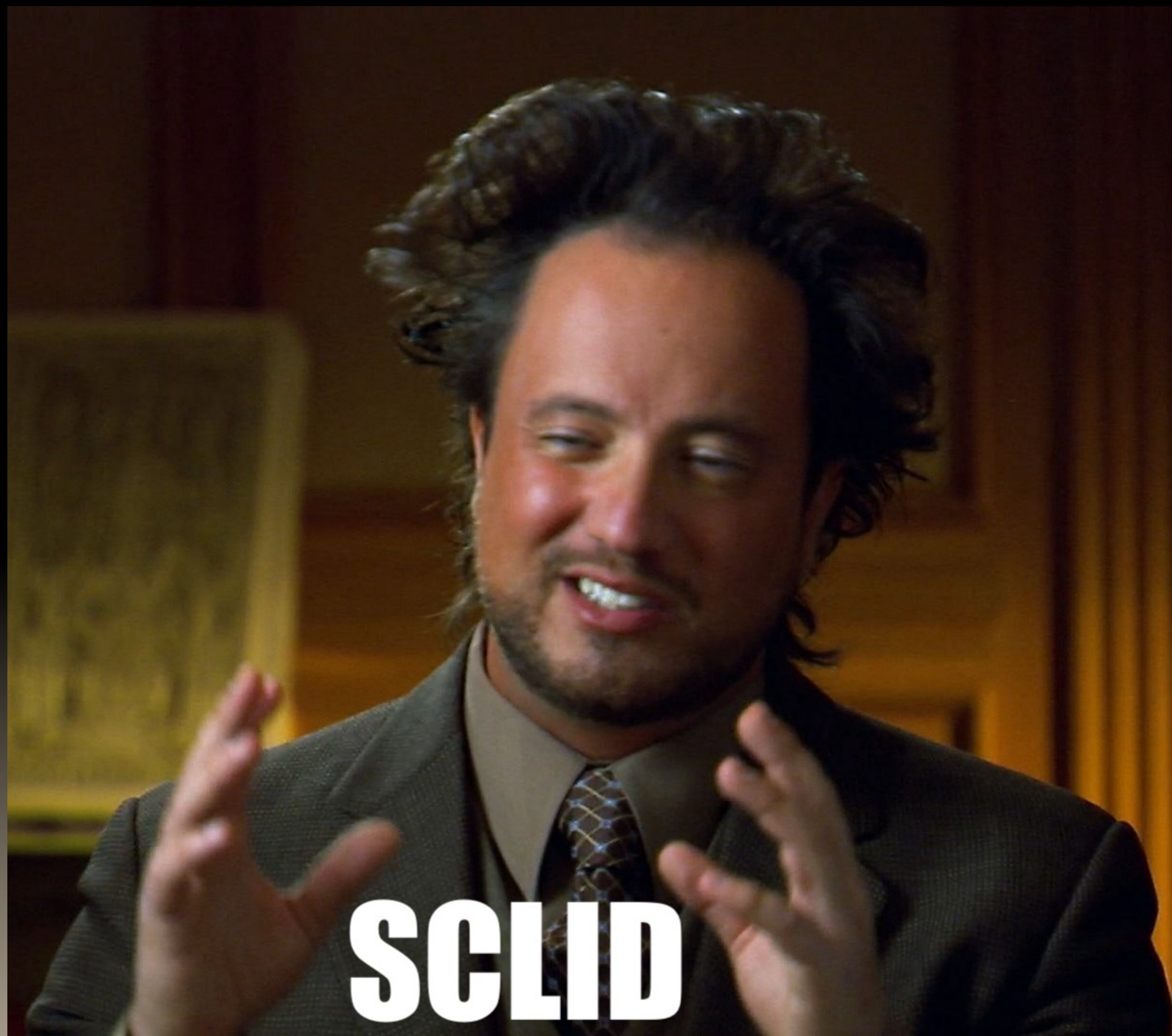
“ Software entities should only be opened for extension when the need arises.



Package maintainer trigger alert



Closed-by-Default ❤️ Open/Closed



In Conclusion...

Use language constructs and patterns to reduce the “cognitive surface” of your code

Be intentional as to where and when you open up your code

Use static analysis for consistency and predictability

In Conclusion...

Value objects, **DateTimeImmutable**, named constructors...

Custom static analysis rules

Architectural static analysis (Deptrac, Pest...)

Guardrails for AI-generated code

“ The Closed-by-Default Principle removes dead ends from your mental maze of code, revealing a straighter path to understanding.





Code and slides

Thank you!



Keep in touch